

Characterizing Speculative Decoding Dynamics for Large Language Models on Consumer-Class GPUs

Anonymous Submission

Author details omitted for double-blind review

Abstract—Speculative decoding is widely reported to improve large language model (LLM) latency, but practical behavior on consumer RTX devices is highly sensitive to hardware profile, runtime path, and decoding policy. We present a controlled study spanning two model families (Qwen2.5 and Qwen3) and two consumer-GPU setups (RTX4090 and RTX5090-laptop). For fixed-policy settings ($k \in \{4, 8, 16\}$; deterministic/stochastic), the RTX4090 Qwen3 run family shows best speedup at $k=4$, deterministic ($S=2.8548$) and monotonic degradation with larger k ($\bar{S}_{k=4} = 2.6673$, $\bar{S}_{k=8} = 1.8258$, $\bar{S}_{k=16} = 1.0405$), while acceptance efficiency and accepted block size move in opposite directions for latency optimization. Across matched pairs, deterministic remains faster than stochastic at every k (mean gap 0.2633 speedup units). On RTX5090-laptop, the same ordering trends persist (best at $k=4$, deterministic; lower at larger k ; deterministic $>$ stochastic), but absolute speedups are below 1 under the tested stack, highlighting a deployment-critical portability gap. We additionally report seed-level variance (mean $\pm$ std), time to first token (TTFT), time per output token (TPOT), and corrected paired tests to strengthen statistical transparency. The Qwen2.5 results show the same qualitative low- k and deterministic-preference trends, providing cross-family consistency at the trend level. These findings refine the paper’s claim: this work provides reproducible, deployment-oriented characterization and policy guidance rather than a universally throughput-improving new algorithm.

Index Terms—speculative decoding, large language models, inference acceleration, benchmarking, reproducibility

I. INTRODUCTION

Autoregressive decoding in large language models (LLMs) is inherently sequential: each generated token requires a full forward pass of the target model. This constraint directly limits single-stream throughput in interactive inference settings. Speculative decoding mitigates this bottleneck by adding a smaller draft model that proposes a length- k token block, followed by one target-side verification step; modified rejection sampling preserves the target distribution [1], [2].

Although speculative decoding is now well studied, deployment guidance remains uncertain for consumer-GPU settings. Reported gains in the literature are often obtained on data-center hardware and are sensitive to model pairing, proposal length, and sampling regime. For practitioners operating on a single RTX-class GPU, the key questions are practical: which draft size is worthwhile, which k is optimal, and whether deterministic versus stochastic decoding changes the speed-quality frontier.

This study addresses three practical gaps for deployment-facing evidence: (i) broader hardware/model validation, (ii) clearer seed-level variance and statistical testing details, and (iii) explicit positioning of Adaptive Cascaded Speculative Decoding (ACSD) as a robustness policy rather than a speedup-maximizing algorithm.

This direction matters for privacy-sensitive and latency-critical local use, including local workstations and edge-side decision-support workflows. We therefore position this paper as a hardware-aware characterization study, rather than a new decoding algorithm.

Within this scope, ACSD is not introduced as a mechanism to exceed the peak throughput of the best static speculative setting. Instead, we position ACSD as a hardware-aware tail-latency guardrail for dynamically difficult inputs, trading a small amount of mean throughput for more controlled runtime behavior under acceptance drops.

A. Research questions

We address four research questions:

- RQ1.** How much end-to-end speedup does vanilla speculative decoding deliver against autoregressive decoding on consumer RTX hardware with Qwen2.5 and Qwen3 family pairings, and how close is this to a practical hardware-constrained upper bound?
- RQ2.** How do draft model size and draft length jointly determine token acceptance and net latency, and what do these trends imply about memory-traffic and kernel-launch overhead on consumer GPUs?
- RQ3.** How sensitive is the speed-quality trade-off to the sampling regime (deterministic vs. stochastic) and to task entropy?
- RQ4.** Do the ranking trends (optimal k , regime preference) transfer across consumer-GPU profiles (desktop RTX4090 vs. RTX5090-laptop), and where do absolute gains fail to transfer?

B. Contributions

This paper makes the following contributions:

- 1) A reproducible consumer-GPU protocol with frozen sample manifests, matched prompts, and two decoding regimes across GSM8K, MMLU subset, and CNN/DailyMail.

- 2) A two-family fixed-policy study combining Qwen2.5 and Qwen3 configurations, with $k \in \{4, 8, 16\}$ and deterministic/stochastic regimes, reported with latency, TTFT, TPOT, acceptance, and quality metrics.
- 3) Cross-hardware validation across desktop RTX4090 and RTX5090-laptop configurations, showing trend portability (best at low k , deterministic preference) but non-portable absolute gains.
 - item Expanded statistical reporting: seed-level mean $\pm$ std summaries, paired tests with Holm-Bonferroni correction, and explicit effect reporting.
 - item An ACSD interpretation and ablation that prioritizes runtime stability and tail behavior over peak throughput claims.

The rest of the paper is organized as follows. Section II reviews related work. Section III defines the experimental protocol; Section IV defines metrics. Section V describes implementation and reproducibility controls. Section VI reports the main empirical findings. Section VII outlines an adaptive speculative-strategy framework and its observed runtime behavior. Section VIII discusses implications, limitations, and conclusions.

II. RELATED WORK

a) Vanilla speculative decoding: Leviathan *et al.* [1] introduced the canonical draft–verify scheme: a smaller *draft* model proposes k tokens autoregressively and the larger *target* verifies them in a single parallel forward pass, with modified rejection sampling guaranteeing that the output distribution is identical to that of the target. Chen *et al.* [2] concurrently proposed an equivalent scheme. The expected per-cycle latency is

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \quad T_{\text{draft}} = k \cdot t_{\text{draft step}}, \quad (1)$$

where $\tau \in [1, k+1]$ is the expected number of accepted tokens per cycle. Wall-clock speedup $S = L_{\text{target}}/L$ therefore depends on the draft/target cost ratio and the empirical token acceptance rate α . Stern *et al.* [3] earlier explored blockwise parallel decoding within a single model.

b) Reducing drafting cost: Medusa [4] eliminates the external draft by attaching multiple prediction heads to the target model and verifying the resulting candidate tree in one forward pass. The EAGLE family [5]–[7] keeps an external draft but conditions it on target hidden features, reporting substantial speedups under their settings. Despite these refinements, T_{draft} remains linear in k , forcing EAGLE-3 to a single transformer layer and capping reported gains at about 2–3 $\times$ on strong GPUs.

c) Diffusion-based drafters: Recent work directly attacks the linear-in- k drafting bottleneck. DiffuSpec [8] and SpecDiff-2 [9] explore diffusion-based parallel drafting to improve speculative-decoding throughput. PARD [10] focuses on a low-cost parallel draft model adaptation. DFlash [11] uses a lightweight 5-layer block-diffusion drafter conditioned on target hidden features and reports up to 5 $\times$ speedup over autoregressive baselines on H200/B200 hardware.

d) Cascaded speculative control: Recent advancements in cascaded architectures attempt to dynamically balance inference efficiency and generation quality. For instance, CAS-Spec constructs a hierarchy of draft stages on-the-fly from a single target model using Dynamically Switchable Inference Acceleration (DSIA) strategies, such as layer sparsity and activation quantization, coordinated by a Dynamic Tree Cascade (DyTC) algorithm [12]. Similarly, Speculative Cascades employs a hybrid approach with a deferral rule, invoking larger, costlier models only for complex inputs while using smaller models for simpler tasks, thereby optimizing the cost-quality trade-off [13]. While these methods report strong acceleration, they typically require target-side architectural modification, additional training, or complex serving infrastructure. In contrast, our ACSD framing is a training-free, architecture-agnostic runtime controller that adapts online decisions from acceptance signals. Its objective is practical runtime stability under constrained consumer-GPU budgets rather than theoretical peak throughput on data-center accelerators.

e) Precision and systems deployment: Deployment-oriented inference work shows that quantization and memory traffic are central to practical throughput gains on constrained hardware. Low-precision techniques such as LLM.int8() [14] reduce weight movement and can interact with speculative decoding in two opposing ways: lower per-step cost for draft/verify passes, but possible shifts in acceptance behavior when verifier and drafter precision settings differ. This interaction motivates reporting speed and quality jointly, rather than throughput alone.

f) Distributed and edge relevance: In distributed and mobile computing settings, local inference can reduce round-trip latency and cloud dependency for interactive workloads. In this context, speculative decoding is relevant beyond chatbot serving, including edge language interfaces where bounded latency and predictable runtime matter as much as peak throughput.

g) Position of this work: This work focuses on a controlled consumer-GPU evaluation of vanilla speculative decoding with same-family model pairing (Qwen3-4B target, 0.6B draft) under deterministic and stochastic regimes. Rather than competing with data-center throughput claims, we characterize practical operating points across desktop RTX4090 and RTX5090-laptop profiles, including acceptance dynamics, quality drift, and runtime-sensitive configuration choices. We then use these results to evaluate adaptive cascaded speculative decoding (ACSD) as a runtime-stability extension.

III. EXPERIMENTAL PROTOCOL

A. Datasets and sample sizes

We evaluate on three benchmarks spanning reasoning, knowledge, and summarization:

- 1) **GSM8K** [15] test subset: 300 samples.
- 2) **MMLU** [16] subset: 500 samples (5 subjects $\times$ 100 each).
- 3) **CNN/DailyMail** [17] subset: 200 samples.

TABLE I
HARDWARE PROFILE FOR THE DESKTOP RTX4090 RUN FAMILY.

Component	Specification
GPU	NVIDIA RTX4090 GPU
GPU memory	24 GB GDDR6X
Memory bandwidth	1008 GB/s
CUDA cores	16384
Architecture	Ada Lovelace
Power profile	TGP class 450 W
Software stack	PyTorch + Transformers (pinned env)

Sampling policy: Fixed random seed = 42, stratified sampling where applicable, and a frozen sample-ID manifest generated before experimentation. Each output CSV records the seed and sample identifier, enabling paired analysis at the sample level.

B. Prompt set

We use the following fixed prompt templates for each task:

GSM8K:

You are a careful math assistant. Solve the problem step by step and end with Final Answer: <number>.
Question: {question}

MMLU:

You are a knowledgeable assistant. Choose exactly one option.
Question: {question}
Options: A. {A} B. {B} C. {C} D. {D}
Answer:

CNN/DailyMail:

Summarize the following article in 3–4 sentences, preserving key facts and entities.
Article: {article}
Summary:

a) Objective and hypotheses: The study quantifies practical speculative-decoding gains on consumer hardware under fixed data and prompt controls. We test four hypotheses:

- **H1:** Same-family speculative decoding yields positive net speedup ($S > 1$) over autoregressive decoding.
- **H2:** Increasing draft length from $k=4$ to $k=8$ improves speedup, while $k=16$ may reduce gains due to additional drafting and verification overhead.
- **H3:** Increasing k tends to improve accepted block size but can reduce end-to-end speed once drafting/verification overhead dominates.
- **H4:** Deterministic decoding is more runtime-efficient and quality stable than stochastic decoding for the same draft and k .

b) Hardware and software controls: This study reports two consumer-GPU run families: an NVIDIA RTX4090 desktop profile and an RTX5090-laptop profile. Both use the same Python runtime entry points, prompt templates, tokenizer settings, and serial execution policy to minimize cross-run contention. We keep software dependencies pinned within each run family and preserve frozen manifests for paired comparison.

c) CUDA-graph runtime setup: To reduce per-step launch overhead, we use a CUDA-graph-capable verify path. For each k , we keep fixed step shapes, pre-allocate static draft/verify buffers, and warm up before optional `torch.cuda.CUDAGraph` capture. During decoding, each cycle updates buffer contents and replays the graph when shape guards hold; otherwise, it falls back to eager execution. This preserves output semantics while reducing host-side overhead in short speculative cycles.

d) Model pairing: We evaluate two same-family pairings. For Qwen2.5, the target is Qwen2.5-3B-Instruct with Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct drafts on the RTX4090 run family. For Qwen3, the target is Qwen3-4B-Instruct with a Qwen3-0.6B-Instruct draft on RTX4090 and RTX5090-laptop run families. For each family, fixed-policy experiments sweep $k \in \{4, 8, 16\}$ under deterministic/stochastic regimes with regime-matched autoregressive baselines. All configurations process fixed manifests to preserve paired comparability.

e) Task-entropy interpretation: We treat the three tasks as distinct decoding-entropy regimes to interpret acceptance behavior: GSM8K is a lower-entropy chain reasoning with concentrated next-token mass; MMLU is medium-entropy constrained QA; CNN/DailyMail is higher-entropy open-ended summarization. This framing is used only for interpretation of acceptance and quality trends, not as an additional optimization objective.

f) Decoding regimes and grid: Two regimes are evaluated:

- **Deterministic:** greedy decoding ($T=0.0$, $\text{top-}p=1.0$; sampling disabled).
- **Stochastic:** sampling with $T=0.7$, $\text{top-}p=0.9$, and no $\text{top-}k$ truncation.

For the primary fixed-policy matrix, we sweep $k \in \{4, 8, 16\}$ under both regimes (6 configurations), plus regime-matched autoregressive baselines.

g) Endpoints: The primary endpoint is paired wall-clock speedup $S = L_{\text{autoregressive}}/L_{\text{spec}}$ as reported by the experiment summary artifact for each draft- k -regime combination. Secondary endpoints include acceptance rate α , effective accepted block size B_{eff} , mean per-sample latency, throughput (tokens/s), task-level quality deltas, time to first token (TTFT), time per output token (TPOT), and 99th-percentile (P99) latency. Statistical reporting uses seed-level summaries (mean $\pm$ std), paired bootstrap confidence intervals where applicable, and paired t-tests with Holm-Bonferroni correction for multi-metric ACSD comparisons.

IV. EVALUATION METRICS

Table II defines the exact reporting metrics used in this study.

A. Success criteria

Primary:

- 1) Mean speedup $S \geq 1.3\times$ with CI_S lower bound > 1.0 .

TABLE II
METRIC DEFINITIONS USED IN THIS STUDY.

Metric	Symbol	Computation	Unit	Direction
End-to-end latency	T	completion_time — request_start	s	Lower
Throughput	R_{tok}	output_tokens / T	tokens/s	Higher
Time-to-first-token	TTFT	first_token_time — request_start	ms	Lower
Time-per-output-token	TPOT	(completion_time — first_token_time) / output_tokens	ms/token	Lower
Acceptance rate	α	accepted_draft_tokens / proposed_draft_tokens	ratio	Higher
Effective block size	B_{eff}	accepted_draft_tokens / verify_steps	tokens/step	Higher
Relative speedup	S	baseline_latency / speculative_latency	$\times$	Higher
GSM8K quality	Q_{gsm}	correct / total	%	Higher
MMLU quality	Q_{mmlu}	correct / total	%	Higher
CNN/DailyMail quality	Q_{sum}	standard ROUGE-L F1	score	Higher
Quality delta	ΔQ	$Q_{\text{spec}} - Q_{\text{base}}$	points	≈ 0 or +
Speedup confidence interval	CI_S	paired bootstrap percentile CI of S (10k resamples)	$\times$	Higher lower-bound
Speedup significance	p_S	one-sided paired test for $H_0 : S \leq 1$	p-value	Lower
Speedup stability	σ_S	std(S over seeds)	$\times$	Lower

2) Absolute quality drop ≤ 1.0 point on GSM8K and MMLU in deterministic regime.

Secondary: Identify best (k , regime) maximizing S while satisfying quality constraints, report seed-level variance (mean $\pm$ std), and test whether ranking trends transfer across hardware profiles even when absolute speedups differ.

V. IMPLEMENTATION AND REPRODUCIBILITY

a) Code organization: The experiment pipeline is implemented under the project source directory with modular components for configuration, data loading, and autoregressive baselines, speculative decoding, metric aggregation, and runtime orchestration. The notebook driver executes these modules in fixed phase order and writes artifacts to the results directory.

b) Model loading and precision: In the current configuration, target and draft models are loaded in 16-bit floating point (FP16) for the main sweep, with quantization controls retained in configuration for alternative runs. The tokenizer and generation limits are kept constant between baselines and speculative runs to preserve paired comparability.

c) Baselines: We run one autoregressive baseline per regime (deterministic and stochastic) with the same manifests and output-length constraints as speculative runs. Each baseline artifact stores per-sample latency, output length, and task predictions.

d) Speculative decoding implementation.: The speculative loop follows standard token-level draft–verify logic. At each cycle, the draft proposes up to k tokens autoregressively; the target verifies the proposal in one pass over the extended prefix; the accepted prefix is committed up to the first mismatch; and one correction token is added from the target distribution when needed. This procedure preserves target-level generation semantics while exposing acceptance statistics needed for analysis. Both draft and target states are tracked with key-value (KV) caches throughout the decode loop. Figure 1 summarizes this implementation flow.

e) CUDA-graph overhead model and observed effect: For a speculative cycle, we model runtime as

$$T_{\text{cycle}} = k t_{\text{draft}} + t_{\text{verify}} + n_{\text{launch}} t_{\text{launch}}, \quad (2)$$

where $n_{\text{launch}} t_{\text{launch}}$ captures host-side kernel-launch overhead. With successful graph replay, the launch cost is reduced to a lower replay overhead term:

$$T_{\text{cycle}}^{\text{graph}} \approx k t_{\text{draft}} + t_{\text{verify}} + n_{\text{replay}} t_{\text{replay}}, \quad t_{\text{replay}} \ll t_{\text{launch}}. \quad (3)$$

In the Qwen3 run families, graph-capture metadata differs by hardware: the RTX4090 fixed-policy matrix reports stable captured execution, whereas RTX5090-laptop sensitivity runs show that graph-enabled toggles do not guarantee end-to-end latency gains under all runtime conditions. This is why we report both absolute speedup and component-level timing rather than assuming graph capture alone explains throughput behavior.

f) Dynamic-scheduling bottleneck in native PyTorch paths: Fallback eager execution should not be viewed only as an implementation miss. It reflects a broader systems challenge for end-to-end dynamic speculation. When acceptance-dependent control flow, sampling branches, and cache updates change per step, preserving lossless semantics often requires CPU-visible orchestration between kernels. Without a deeper runtime/kernel redesign, this serial control overhead can erase expected gains from speculative parallelism, as also emphasized in modern inference runtime roadmaps for dynamic scheduling.

g) Tokenizer isomorphism as an enabling design choice: Same-family draft/target pairing is a central engineering decision in this study. Because the draft and target tokenizers are isomorphic, speculative verification avoids per-step vocabulary alignment and token translation. Cross-tokenizer pipelines introduce additional CPU-side mapping overhead that can materially reduce or negate speculative speedup on constrained hardware.

h) Regimes and outputs: The same verification core is used for deterministic and stochastic modes, with only

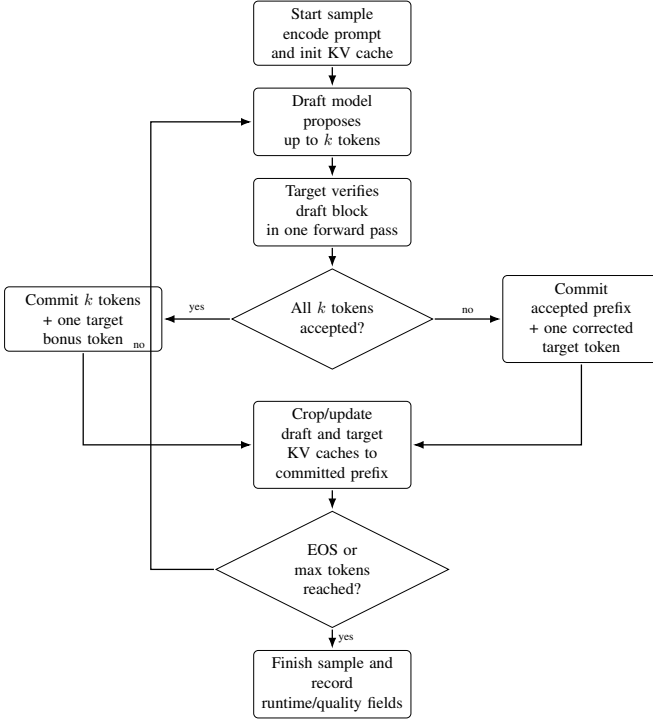


Fig. 1. Vanilla speculative decoding implementation flow. The target model remains the verifier of record; accepted draft prefixes are committed, and caches are cropped to preserve exact target-consistent decoding.

the sampling policy changed by regime parameters. Each configuration writes a dedicated CSV artifact with per-sample runtime, token counts, and quality-relevant outputs.

VI. RESULTS

This section reports fixed-policy results for Qwen2.5 and Qwen3 families across two consumer-GPU setups: desktop RTX4090 and RTX5090-laptop. We focus on paired speedup S , acceptance dynamics (α , B_{eff}), and token-timing metrics (time to first token, TTFT; time per output token, TPOT), then summarize portability implications.

A. RQ1: Does speculative decoding accelerate inference?

On RTX4090, five of six fixed-policy configurations are faster than baseline, with best speedup at 0.6B, $k=4$, deterministic ($S=2.8548$; Table III). The single below-baseline point is high- k stochastic ($k=16$, $S=0.9719$), which indicates that speculative decoding is not uniformly beneficial even within one hardware family when proposal length is too large.

B. RQ2: How do k and acceptance affect net latency?

The results retain the same systems trend: larger k increases accepted block size but degrades end-to-end speed. On RTX4090, $\bar{S}_{k=4} = 2.6673$, $\bar{S}_{k=8} = 1.8258$, $\bar{S}_{k=16} = 1.0405$, while B_{eff} increases from about 1.2 to about 1.8. This confirms an overhead inflection: acceptance efficiency falls with k (α : 0.3362/0.2672 at $k=4$ to 0.1292/0.1082 at $k=16$), and the extra drafting/verification work dominates any block-size benefit.

C. RQ3: Deterministic versus stochastic regime

Deterministic decoding is faster than stochastic at every matched k in the reported matrix: $\Delta S_{k=4}=0.3750$, $\Delta S_{k=8}=0.2779$, $\Delta S_{k=16}=0.1372$ (deterministic minus stochastic), with mean gap 0.2633 speedup units. TTFT and TPOT also show consistent deterministic advantage (Table III).

D. RQ4: Cross-hardware portability (RTX4090 vs RTX5090-laptop)

The ordering pattern transfers across devices (low- k best, deterministic preferred, and monotonic decline with larger k), but absolute gains do not. Under the tested RTX5090-laptop software/runtime stack, all reported fixed-policy points are slower than autoregressive baseline. This indicates that trend-level conclusions are more portable than absolute speedup magnitudes.

E. Variance, TTFT/TPOT, and statistical testing detail

To provide stronger statistical detail, we include seed-level summaries from repeated fixed- k runs. Across seeds, deterministic speedup means are 2.2876 ± 0.0242 ($k=4$), 2.2712 ± 0.0295 ($k=8$), and 2.0511 ± 0.0273 ($k=16$); stochastic means are 1.8910 ± 0.0220 , 1.9055 ± 0.0257 , and 1.6698 ± 0.0133 . TTFT is stable (about 50–52 ms), while TPOT and 99th-percentile (P99) latency rise with larger k , consistent with the overhead interpretation.

We also report paired tests with Holm-Bonferroni correction for ACSD-versus-fixed comparisons (Section VII), separating statistically significant token-timing effects from non-significant quality differences in that comparison set.

F. Summary of practical operating point

For the Qwen3 evidence, the most reliable deployment recommendation on RTX4090 remains deterministic $k=4$ as the default fast path. Cross-device validation supports this policy ranking but warns that hardware/runtime portability is limited for absolute throughput gains.

G. Cross-family consistency (Qwen2.5 vs Qwen3)

To address model-family applicability, we compare Qwen2.5 RTX4090 results with the Qwen3 results. The absolute speedup scale differs across runs, but the core qualitative patterns are consistent across both families on RTX4090: (i) best operating point at low k (especially $k=4$), (ii) worsening speedup as k increases, and (iii) deterministic preference in most matched settings.

Concretely, the Qwen2.5 table reports best fixed speedup at 0.5B, $k=4$, deterministic ($S=3.0674$), with mean- k trend $\bar{S}_{k=4} = 2.9164 > \bar{S}_{k=8} = 2.4029 > \bar{S}_{k=16} = 1.8019$. In the Qwen3 RTX4090 run family, we observe the same ordering with different magnitudes ($\bar{S}_{k=4} = 2.6673 > \bar{S}_{k=8} = 1.8258 > \bar{S}_{k=16} = 1.0405$).

These aligned trend-level findings support a stronger but still bounded claim: the operating-policy ranking appears

TABLE III
QWEN3 FIXED-POLICY RESULTS ON RTX4090 AGAINST REGIME-MATCHED BASELINES. SPEEDUP IS COMPUTED AS A RATIO OF MEANS AGAINST THE CORRESPONDING BASELINE.

Config	S	α	B_{eff}	T_{mean} (s)	TTFT (ms)	TPOT (ms)
0.6B, $k=4$, det	2.8548	0.3362	1.33	4.8881	132.57	41.23
0.6B, $k=4$, stoch	2.4798	0.2672	1.06	5.6196	134.54	45.51
0.6B, $k=8$, det	1.9648	0.2378	1.86	7.1024	204.99	58.76
0.6B, $k=8$, stoch	1.6869	0.1916	1.50	8.2612	207.82	64.67
0.6B, $k=16$, det	1.1091	0.1292	1.96	12.5817	349.58	94.24
0.6B, $k=16$, stoch	0.9719	0.1082	1.64	14.3390	354.82	106.03

TABLE IV
QWEN3 FIXED-POLICY RESULTS ON RTX5090-LAPTOP. ORDERING TRENDS ARE PRESERVED (BEST AT LOW k , DETERMINISTIC > STOCHASTIC), BUT ABSOLUTE SPEEDUPS ARE BELOW 1 IN THIS RUN FAMILY.

Config	S	α	B_{eff}
0.6B, $k=4$, det	0.6743	0.3353	1.32
0.6B, $k=4$, stoch	0.5924	0.2845	1.06
0.6B, $k=8$, det	0.4613	0.2538	1.85
0.6B, $k=8$, stoch	0.3995	0.2273	1.50
0.6B, $k=16$, det	0.2588	0.1803	1.96
0.6B, $k=16$, stoch	0.2222	0.1543	1.64

more portable across these two model families than absolute throughput values, which remain implementation- and hardware-path dependent.

VII. DISCUSSION ON ADAPTIVE SPECULATIVE STRATEGIES DERIVED FROM EMPIRICAL FINDINGS

The fixed-policy results preserve the key systems signal: low- k configurations dominate high- k settings in end-to-end runtime, while acceptance-related observables alone are insufficient to predict wall-clock wins. We therefore keep ACSD as a control-policy extension intended for runtime stability and policy robustness, not as a guaranteed peak-throughput replacement for the best fixed setting.

A. Design objective

Let speculative-cycle latency be

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \quad (4)$$

where τ is the expected accepted tokens per cycle. ACSD targets adverse samples where τ collapses for sustained windows, by adapting policy online (draft length, rescue behavior, and fallback rules) rather than fixing a single global k .

B. Controller formalization

We model ACSD as a finite-state controller with states $\{\mathcal{D}_{\text{pri}}, \mathcal{D}_{\text{res}}, \mathcal{M}_{\text{AR}}\}$ (primary draft, rescue draft, autoregressive (AR) fallback), and transition rules based on recent acceptance statistics. Four mechanisms are retained: (i) bounded adaptive k_t updates, (ii) rescue switching after repeated low-acceptance windows, (iii) guarded fallback for persistent collapse, and (iv) checkpointed execution for recoverability.

C. Empirical interpretation

Using the ablation tables, ACSD speedup depends on threshold policy. In deterministic mode, representative settings span roughly $S \approx 2.15$ – 2.36 ; in stochastic mode, $S \approx 1.76$ – 1.96 . Compared against fixed- k references, ACSD can be competitive in some parameter regions but is not uniformly superior across all quality and latency criteria.

Paired tests with Holm-Bonferroni correction show that, in the reported comparison set, token-timing differences can be significant while aggregate quality deltas are not consistently significant after correction. This supports a conservative deployment claim: ACSD is a policy instrument for stability trade-offs, not a universal speedup guarantee.

D. How this guidance should be used

For deployment, fixed deterministic $k=4$ remains the default first choice when maximizing mean throughput is the sole objective on RTX4090-class runs. ACSD is appropriate when operators prefer an adaptive policy that can reduce risk from difficult inputs and parameter-sensitive runtime drift, accepting that gains are scenario-dependent.

E. Scope in this paper

Quantitative ACSD claims in this paper are limited to the reported Qwen3-family artifacts and ablation tables. Broader claims across model families, longer contexts, and additional hardware still require dedicated validation.

VIII. DISCUSSION

A. Main empirical takeaways

The Qwen2.5 and Qwen3 evidence supports three robust observations. First, on RTX4090, speculative decoding can provide substantial acceleration, but not for every setting; high- k stochastic can drop below baseline. Second, speedup declines as k increases, even when accepted block size grows. Third, deterministic decoding consistently outperforms stochastic decoding for matched k in both run families.

The cross-hardware extension adds an important qualification. While policy ordering trends transfer from RTX4090 to RTX5090-laptop, absolute speedup magnitudes do not. This supports a practical interpretation: generalization claims should emphasize portable trends and decision rules, not fixed throughput numbers.

B. Novelty and contribution scope

This paper’s contribution is primarily empirical and systems-oriented. We do not claim a new speculative decoding algorithm that universally improves throughput. Instead, we provide a controlled, reproducible characterization that links acceptance dynamics, token timing, and latency tails to practical operating choices on consumer hardware.

C. Variance and evaluation depth

To strengthen rigor, we report seed-level mean $\pm$ std summaries, time to first token (TTFT), time per output token (TPOT), 99th-percentile (P99) latency, and corrected paired tests. This improves repeated-run variability reporting and statistical clarity. We also expand quality reporting beyond a single summarization metric in the ablation artifacts (ROUGE-L, BLEU, and BERTScore), while acknowledging that these still do not fully capture factuality or task-specific utility.

IX. THREATS TO VALIDITY

Hardware and runtime dependence. Results remain sensitive to implementation stack and runtime path. The RTX4090 and RTX5090-laptop comparison demonstrates that ranking trends can transfer while absolute gains diverge.

Model-family scope. This study centers on same-family Qwen2.5 and Qwen3 target/draft pairings. Cross-family speculative pairings may show different acceptance and quality behavior.

Evaluation breadth. The benchmark suite covers reasoning, knowledge QA, and summarization, but does not exhaust long-context, safety, or factual consistency requirements.

X. CONCLUSION

This paper presents reproducible deployment guidance for consumer GPUs using same-family speculative decoding in both Qwen2.5 and Qwen3 settings. On RTX4090, deterministic $k=4$ remains the strongest operating point family-wise, with monotonic degradation as k increases. Cross-hardware validation on RTX5090-laptop preserves the same policy ordering trend but not the same absolute speedups, emphasizing that portability must be interpreted at the trend level.

ACSD remains useful as an adaptive policy layer for runtime-stability trade-offs, but is not presented as a universal throughput optimizer. Future work will expand hardware diversity, include additional model families, and evaluate longer-context and richer quality diagnostics.

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2211.17192>
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating large language model decoding with speculative sampling,” in *arXiv preprint arXiv:2302.01318*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2302.01318>
- [3] M. Stern, N. Shazeer, and J. Uszkoreit, “Blockwise parallel decoding for deep autoregressive models,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. [Online]. Available: <https://doi.org/10.48550/arXiv.1811.03115>
- [4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple LLM inference acceleration framework with multiple decoding heads,” *arXiv preprint arXiv:2401.10774*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2401.10774>
- [5] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE: Speculative sampling requires rethinking feature uncertainty,” *arXiv preprint arXiv:2401.15077*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2401.15077>
- [6] —, “EAGLE-2: Faster inference of language models with dynamic draft trees,” *arXiv preprint arXiv:2406.16858*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2406.16858>
- [7] —, “EAGLE-3: Scaling up inference acceleration of large language models via training-time test,” *arXiv preprint arXiv:2503.01840*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2503.01840>
- [8] G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang, “DiffuSpec: Unlocking diffusion language models for speculative decoding,” *arXiv preprint arXiv:2510.02358v1*, 2025. <https://arxiv.org/abs/2510.02358v1>. [Online]. Available: <https://doi.org/10.48550/arXiv.2510.02358>
- [9] J. Sandler, J. K. Christopher, T. Hartvigsen, and F. Fioretto, “SpecDiff-2: Scaling diffusion drafter alignment for faster speculative decoding,” *arXiv preprint arXiv:2511.00606*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2511.00606>
- [10] Z. An, H. Bai, Z. Liu, D. Li, and E. Barsoum, “PARDo: Accelerating llm inference with low-cost parallel draft model adaptation,” *arXiv preprint arXiv:2504.18583*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2504.18583>
- [11] J. Chen, Y. Liang, and Z. Liu, “DFlash: Block diffusion for flash speculative decoding,” *arXiv preprint arXiv:2602.06036*, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2602.06036>
- [12] Z. Ning, J. Shao, R. Xu, X. Guo, J. Zhang, C. Zhang, and X. Li, “CAS-Spec: Cascade adaptive self-speculative decoding for on-the-fly lossless inference acceleration of llms,” *arXiv preprint arXiv:2510.26843*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2510.26843>
- [13] H. Narasimhan, W. Jitkrittum, A. S. Rawat, S. Kim, N. Gupta, A. K. Menon, and S. Kumar, “Faster cascades via speculative decoding,” in *Proceedings of the IEEE Conference (arXiv preprint)*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.19261>
- [14] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2208.07339>
- [15] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021. [Online]. Available: <https://doi.org/10.48550/arXiv.2110.14168>
- [16] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2020. [Online]. Available: <https://doi.org/10.48550/arXiv.2009.03300>
- [17] S. Narayan, S. B. Cohen, and M. Lapata, “Don’t give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018. [Online]. Available: <https://doi.org/10.18653/v1/D18-1206>